



Penetration Testing Internship

Submitted By	Irsa Attique
Submitted To	DEV SHIELDX
Date	16/07/2026
Intern ID	DSX-CYB-10
Task	15(Capstone Project)
Batch	2

Capstone Project Report

Target Application: Damn Vulnerable Web Application (DVWA)

Assessment Type: Black-Box Web Application Penetration Test

1. Executive Summary:

To identify misconfigurations, flaws and to exploit vulnerabilities, and independent black box security assessment was conducted against DVWA. The assessment adhered strictly to the industry-standard OWASP Web Security Testing Guide (WSTG) methodology.

The security evaluation uncovered critical vulnerabilities that allow complete underlying system compromise and unauthorized data extraction. Notably, Remote Code Execution (RCE) via Command Injection was verified, alongside Database Credential Disclosure via SQL Injection. Additionally, severe administrative directory listings, lack of defensive HTTP headers, and exposed sub-version control files were discovered.

Immediate code-level remediation is required to secure the target application against real-world unauthorized access and operational disruptions.

2. Objective:

The objective of this task was to:

- Gain access to the application's posture from an external threat perspective.
- Enumerate crawl directories, enumerate services and physically validate security flaws without disrupting service availability.
- Gauge the severity of validated findings using the Common Vulnerability Scoring System (CVSS v3.1).
- Outline precise, programmatic steps to patch and protect the target application code from exploitation.

3. Scope:

The security assessment was bounded to the local deployment instance of the DVWA environment:

- **Target URL:** http://localhost/DVWA/
- **Target IP Address:** 127.0.0.1 (Localhost)
- **Testing Modality:** Black-Box Testing.

- **Constraints:** Active exploits were executed exclusively against the self-hosted, authorized lab environment to maintain strict compliance and prevent lateral real-world impacts.

4. Methodology:

Standardized phases of the OWASP Web Security Testing Guide (WSTG) were followed during the assessment which utilized an organized five-stage lifecycle:

- **Reconnaissance:** Active footprinting of the target server, identifying open ports, active services, and technology signatures using “WhatWeb and Nmap”.
- **(Enumeration):** Spidering and directory-busting using “FFUF and Nmap NSE” scripts to discover exposed directories, hidden administrative endpoints, and orphaned source control configurations.
- **Vulnerability Assessment:** Analyzing structural weaknesses and identifying missing secure headers or weak platform parameters using “Nikto”.
- **Exploitation:** Executing proof-of-concept (PoC) payloads targeting Command Injection and SQL Injection endpoints, demonstrating the real-world impact of the flaws.
- **Reporting & Remediation:** Documenting all technical findings, calculating CVSS impact, and outlining exact patch instructions for stakeholders.

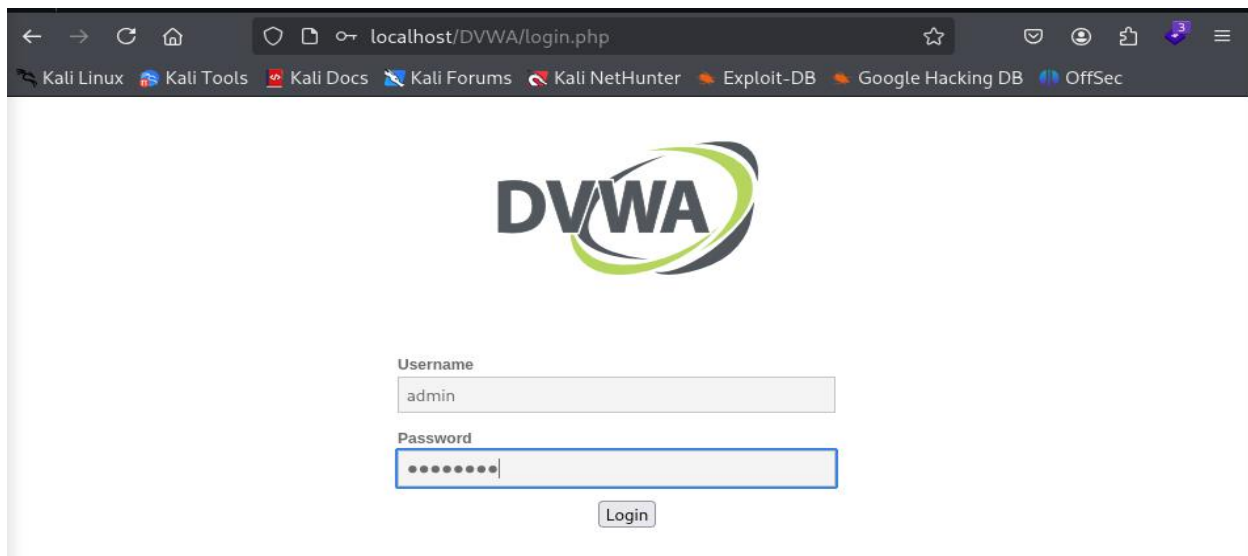
5. Environment Setup:

```
(kali@kali)-[~/Desktop]
$ sudo systemctl start apache2

(kali@kali)-[~/Desktop]
$ sudo systemctl start mysql
```



(Initiating Apache and MySQL system services in the Kali Linux environment.)



(Navigating to the local DVWA landing login page.)




Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Low

▼

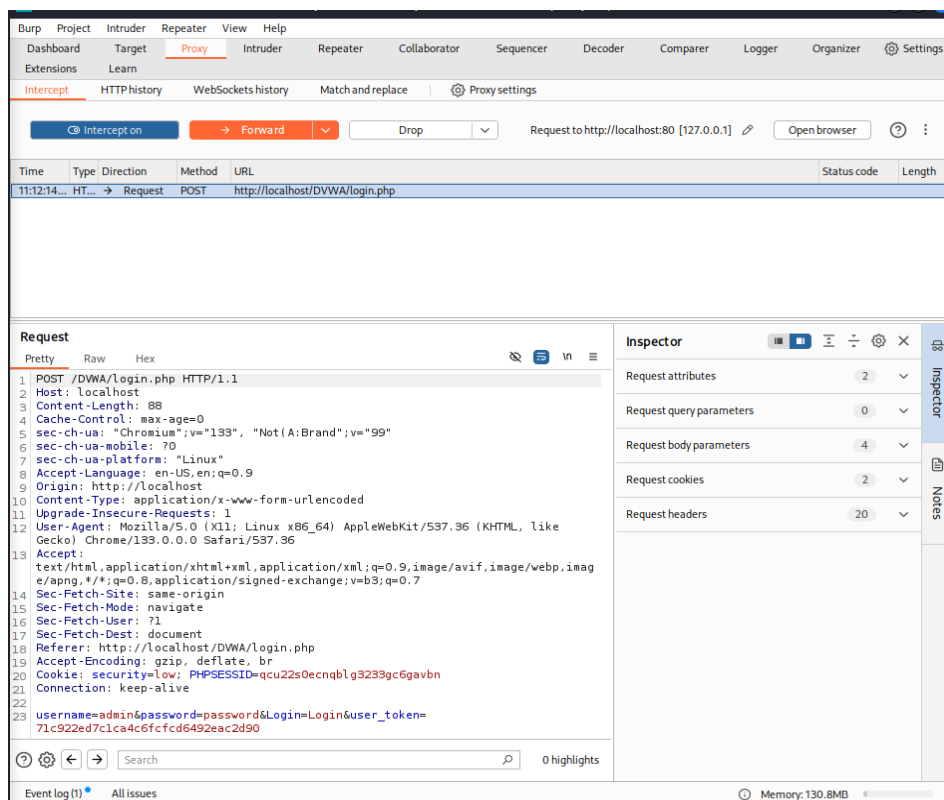
Submit

Additional Tools

- [View Broken Access Control Logs](#) - View access logs for the Broken Access Control vulnerability

Security level set to low

(Setting the target testing baseline security setting to 'low'.)

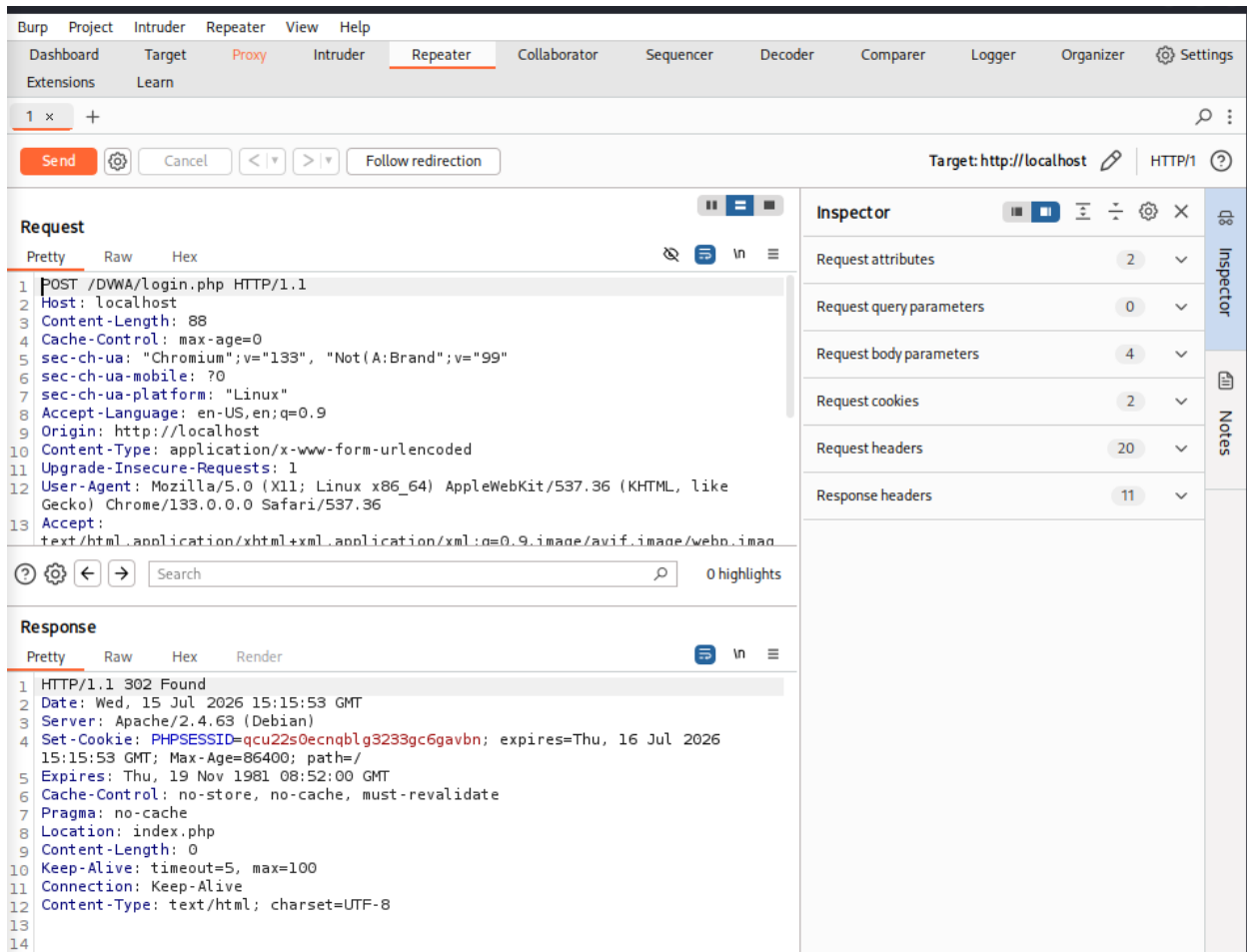


The screenshot shows the Burp Suite interface with the 'Proxy' tab selected. A request is captured and displayed in the 'Request' pane. The request is a POST to `http://localhost/DVWA/login.php`. The 'Inspector' pane on the right shows the request details, including headers and cookies. The cookies section shows `security=low` and `PHPSESSID=qcu22s0ecnqlg3233gc6gavbn`.

```

1 POST /DVWA/login.php HTTP/1.1
2 Host: localhost
3 Content-Length: 88
4 Cache-Control: max-age=0
5 sec-ch-ua: "Chromium";v="133", "Not(A:Brand";v="99"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost/DVWA/login.php
19 Accept-Encoding: gzip, deflate, br
20 Cookie: security=low; PHPSESSID=qcu22s0ecnqlg3233gc6gavbn
21 Connection: keep-alive
22
23 username=admin&password=password&Login=Login&user_token=71c922ed7d1ca4c6fcfd6492eac2d90
  
```

(Capturing raw authentication POST parameters and CSRF tokens using the Burp Suite Intercept proxy.)



(Analyzing request authentication flow and the redirect response status code using Burp Suite Repeater.)

6. Target Information:

Baseline technology footprinting was performed using the whatweb CLI utility to determine the server configuration and underlying technologies.

Fingerprinting Signature Output (whatweb http://localhost/DVWA/login.php)

- **Target Endpoint:** http://localhost/DVWA/login.php
- **HTTP Status Code:** 200 OK
- **Web Server:** Apache HTTP Server version 2.4.63 (Debian)
- **Operating System Platform:** Debian Linux
- **Session Cookie Engine:** PHPSESSID

- **Authentication/Security Parameter:** security cookie parameter tracking active security levels.
- **Markup/Interactivity:** HTML5 compliant webpage with input password fields.

```
(kali@kali)-[~/Desktop]
$ whatweb http://localhost/DVWA/login.php
http://localhost/DVWA/login.php [200 OK] Apache[2.4.63], Cookies[PHPSESSID,security], DVWA, HTML5, HTTPServer[Debian Linux][Apache/2.4.63 (Debian)], IP[::1], PasswordField[password], Title[Login :: Damn Vulnerable Web Application (DVWA)]
```

(Target technology footprinting signature output using WhatWeb)

7. Reconnaissance Findings:

Nmap was used for active scanning which helped to map accessible transport layers and services listening on the target.

Port and Service Scan (nmap -sV -p 80,3306 localhost)

The system runs a LAMP/WAMP equivalent environment. Port 80 handles public web traffic, and Port 3306 hosts the backend MariaDB database. Because Port 3306 is open, it represents a potential vector for direct database connectivity attacks if firewalling is inadequate.

```
(kali@kali)-[~/Desktop]
$ nmap -sV -p 80,3306 localhost
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-15 10:48 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00019s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.63 ((Debian))
3306/tcp  open  mysql   MariaDB 11.4.5

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.84 seconds
```

(Nmap service scanning confirming open ports 80 (HTTP) and 3306 (MySQL).)

Passive Directory Enumeration (nmap --script http-enum localhost)

The administrative /server-status/ folder is exposed. This directory can leak active client IP addresses, request URIs, and server load metrics to unauthenticated users.

```
(kali@kali)-[~/Desktop]
$ nmap --script http-enum localhost
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-15 10:49 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000038s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
| http-enum:
|_ /server-status/: Potentially interesting folder
3306/tcp  open  mysql

Nmap done: 1 IP address (1 host up) scanned in 1.51 seconds
```

(Passive directory enumeration script revealing sensitive system directories.)

8. Vulnerability Assessment:

An automated vulnerability scan using Nikto combined with directory fuzzing via FFUF revealed massive structural flaws in the host configuration.

Automated Vulnerability:

- Missing Anti-Clickjacking Header: X-Frame-Options is not set, exposing users to frame-based UI redressing attacks.
- Missing MIME Sniffing Mitigation: X-Content-Type-Options is not set, allowing browsers to interpret responses away from the declared content-type.
- Insecure Cookie Flags: Session cookies (PHPSESSID) are set without the HTTP Only flag, making them extractable via Cross-Site Scripting (XSS).
- Information Leakage via Directory Indexing:
 - /DVWA/config/ (Configuration files accessible)
 - /DVWA/database/ (Database storage schemas exposed)
 - /DVWA/docs/ (System documentation accessible)
 - /DVWA/tests/ (Test files available)
- Exposed Git Repositories:
 - /DVWA/. git/index
 - /DVWA/. git/HEAD
 - /DVWA/. git/config
 - /DVWA/. git/gitignore

```
(kali@kali) [~/Desktop]
$ nikto -h http://localhost/DVWA/
- Nikto v2.5.0

+ Target IP: 127.0.0.1
+ Target Hostname: localhost
+ Target Port: 80
+ Start Time: 2026-07-15 10:51:14 (GMT-4)

+ Server: Apache/2.4.63 (Debian)
+ /DVWA/: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /DVWA/: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ /DVWA/: Cookie security created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ /DVWA/: Cookie PHPSESSID created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ Root page /DVWA redirects to: login.php
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ OPTIONS: Allowed HTTP Methods: GET, POST, OPTIONS, HEAD .
+ /DVWA/config/: Directory indexing found.
+ /DVWA/config/: Configuration information may be available remotely.
+ /DVWA/tests/: Directory indexing found.
+ /DVWA/tests/: This might be interesting.
+ /DVWA/database/: Directory indexing found.
+ /DVWA/database/: Database directory found.
+ /DVWA/docs/: Directory indexing found.
+ /DVWA/login.php: Admin login page/section found.
+ /DVWA/.git/index: Git Index file may contain directory listing information.
+ /DVWA/.git/HEAD: Git HEAD file found. Full repo details may be present.
+ /DVWA/.git/config: Git config file found. Infos about repo details may be present.
+ /DVWA/.gitignore: .gitignore file found. It is possible to grasp the directory structure.
+ /DVWA/.dockerignore: .dockerignore file found. It may be possible to grasp the directory structure and learn more about the site.
+ 7850 requests: 0 error(s) and 18 item(s) reported on remote host
+ End Time: 2026-07-15 10:51:51 (GMT-4) (37 seconds)

+ 1 host(s) tested
```

Directory Fuzzing Analysis:

A directory search confirmed the open indexing paths found by Nikto:

Heading	Status, Size
.git/HEAD	[Status: 200, Size: 23]
config	[Status: 301, Size: 312]
database	[Status: 301, Size: 314]
docs	[Status: 301, Size: 310]
php.ini	[Status: 200, Size: 154]
robots.txt	[Status: 200, Size: 25]

Exposed. git structures allow an attacker to reconstruct the entire application source code locally, uncovering hardcoded database credentials, private API endpoints, and business logic flaws.

```
(kali@kali)~[~/Desktop]
$ ffuf -u http://localhost/DVWA/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200,301,302

Host: localhost
Content-Type: application/javascript
Cache-Control: no-store, no-cache, max-age=0
sec-ch: ...
sec-ch: ...
Accept: */*
Origin: http://localhost
Content-Type: application/x-www-form-urlencoded
Upgrade: 1
Request: 1
v2.1.0-dev
Server: Apache/2.4.63 (Ubuntu)
:: Method : GET
:: URL : http://localhost/DVWA/FUZZ
:: Wordlist : FUZZ: /usr/share/wordlists/dirb/common.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout : 10
:: Threads : 40
:: Matcher : Response status: 200,301,302
HTTP/1.1 302 Found

Server: Apache/2.4.63 (Ubuntu)
Content-Type: text/html
Connection: Keep-Alive
:: Progress: [4614/4614] :: Job [1/1] :: 64 req/sec :: Duration: [0:00:04] :: Errors: 0 ::
```

(FFUF directory fuzzing confirming directory exposure and sensitive file paths.)

9. Exploitation Results:

The vulnerabilities discovered during the assessment were manually exploited to prove the risk of system compromise.

Vulnerability 1: Remote Code Execution (RCE) via OS Command Injection

- Vulnerable Endpoint: <http://localhost/DVWA/vulnerabilities/exec/>
- Injection Parameter: ip
- Vulnerability Description: The application concatenates user-supplied input into a shell execution context without sanitizing or escaping system shell metacharacters.
- Exploitation Vector: Inputting 127.0.0.1 && whoami causes the system to run the initial ping command followed immediately by the system shell command whoami.

[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)
[File Upload](#)
[Insecure CAPTCHA](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[Weak Session IDs](#)
[XSS \(DOM\)](#)
[XSS \(Reflected\)](#)

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.050 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.102 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.082 ms  
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.034 ms  
  
--- 127.0.0.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3049ms  
rtt min/avg/max/mdev = 0.034/0.067/0.102/0.026 ms
```

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

(Baseline test validating standard ping functionality on the Command Injection target page.)

- Validated Impact: The application successfully executed the payload and returned www-data in the HTTP response. This confirms that the web application possesses shell execution privileges, enabling complete host takeover.



[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)
[File Upload](#)
[Insecure CAPTCHA](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[Weak Session IDs](#)
[XSS \(DOM\)](#)
[XSS \(Reflected\)](#)
[XSS \(Stored\)](#)
[CSP Bypass](#)

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.046 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.052 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.070 ms  
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.155 ms  
  
--- 127.0.0.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3060ms  
rtt min/avg/max/mdev = 0.046/0.080/0.155/0.043 ms  
www-data
```

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

Vulnerability 2: Database Credential Disclosure via SQL Injection

- Vulnerable Endpoint: <http://localhost/DVWA/vulnerabilities/sqli/>
- Injection Parameter: id
- Vulnerability Description: The parameter lacks input parameterization or sanitization, passing raw inputs straight into the SQL interpreter.
- Exploitation Vector: Utilizing a custom SQL UNION payload: 1' UNION SELECT user, password FROM users#

The screenshot shows the 'Vulnerability: SQL Injection' page. On the left is a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection (highlighted), and SQL Injection (Blind). The main content area has a 'User ID:' input field with '1' entered and a 'Submit' button. Below the input, the output displays: ID: 1, First name: admin, Surname: admin. Underneath, a 'More Information' section lists four links related to SQL injection.

(Baseline query validation using ID '1' on the SQL Injection target page.)

- Validated Impact: The backend database executed the query, dumping usernames and their corresponding MD5 password hashes directly onto the page:
 - admin: 5f4dcc3b5aa765d61d8327deb882cf99 (Decodes to password)
 - gordonb: e99a18c428cb38d5f260853678922e03 (Decodes to abc123)
 - 1337: 8d3533d75ae2c3966d7e0d4fcc69216b (Decodes to charley)

This screenshot shows the same 'Vulnerability: SQL Injection' page, but with a more comprehensive list of results. The 'User ID:' field contains a long UNION payload. The output displays six rows of data: ID, First name, and Surname for each. The results are: 1' UNION SELECT user, password FROM users# (admin), 1' UNION SELECT user, password FROM users# (gordonb), 1' UNION SELECT user, password FROM users# (1337), 1' UNION SELECT user, password FROM users# (charley), 1' UNION SELECT user, password FROM users# (pablo), and 1' UNION SELECT user, password FROM users# (smithy). The 'More Information' section remains the same as in the previous screenshot.

10. RISK RATINGS (CVSS v3.1):

Below is the risk rating breakdown based on the verified severity of the findings:

1. Remote Code Execution (Command Injection)

- CVSS v3.1 Base Score: 9.8 Critical
- Vector String: CVSS:3.1/AV: N/AC: L/PR: N/UI: N/S: U/C:H/I:H/A:H
- Risk Level: Critical
- Justification: Can be exploited remotely over the network with low complexity and no special privileges, resulting in a total loss of confidentiality, integrity, and availability.

2. SQL Injection (Credential Disclosure)

- CVSS v3.1 Base Score: 8.6 High
- Vector String: CVSS:3.1/AV: N/AC: L/PR: N/UI: N/S:C/C:H/I: N/A: N
- Risk Level: High
- Justification: Allows unauthenticated actors to read arbitrary database contents, exposing system-wide administrative passwords and user hashes.

3. Exposed Git Repository & Directory Indexing

- CVSS v3.1 Base Score: 7.5 High
- Vector String: CVSS:3.1/AV: N/AC: L/PR: N/UI: N/S: U/C:H/I: N/A: N
- Risk Level: High
- Justification: Public exposure of .git directories and configurations leak vital infrastructure architecture and application source code.

11. Remediation Recommendations:

Remediation for OS Command Injection (RCE)

- Implementation: Never pass raw user inputs directly into system shells (such as PHP's `shell_exec()`, `exec()`, or `passthru()`).
- Code Fix: Use built-in APIs instead of system commands, or apply strict character whitelisting to validate inputs.

Remediation for SQL Injection

- Implementation: Always use Parameterized Queries (Prepared Statements) with PDO or MySQLi. This keeps user inputs separated from the SQL execution logic.

Remediation for Directory Listing and Exposed Git Repositories

- Apache Configuration Fix: Disable directory listings by adding the following rule to your global Apache configuration or localized .htaccess file:
 - Options -Indexes
- Git Path Protection: Delete or restrict web permissions on .git folders in production:
 - `rm -rf /var/www/html/DVWA/.git`

Remediation for Missing Security Headers

Add these secure headers inside the main virtual host directory setup file:

- Header set X-Frame-Options "DENY"
- Header set X-Content-Type-Options "nosniff"
- Header edits Set-Cookie ^ (. *) \$ "\$1; HTTP Only; Secure"

12. Conclusion:

The security assessment of the DVWA revealed that the application has a high-risk profile. The validation of Remote Code Execution and SQL Injection demonstrates that how easily an attacker can extract user database records and execute commands on the underlying host.

Implementing input validation, parameterizing SQL statements, disabling web-accessible directory indices, and securing cookie flags will drastically reduce the application's attack surface and prevent these critical compromises.

13. References:

- OWASP Web Security Testing Guide (WSTG):

<https://owasp.org/www-project-web-security-testing-guide/>

- CWE-78: Improper Neutralization of Special Elements used in an OS Command (OS Command Injection):

<https://cwe.mitre.org/data/definitions/78.html>

- CWE-89: Improper Neutralization of Special Elements used in an SQL Command (SQL Injection):

<https://cwe.mitre.org/data/definitions/89.html>

- CVSS v3.1 Calculator (FIRST):

<https://www.first.org/cvss/calculator/3.1>

14. Internship Reflection:

My cybersecurity internship with Dev ShieldX has been a defining step in my professional journey. As a female student pursuing a BS in Cyber Security at COMSATS University Islamabad, bridging the gap between theoretical classroom concepts and hands-on laboratory application has been incredibly rewarding.

This capstone project consolidated multiple testing workflows: passive infrastructure fingerprinting, automated vulnerability assessments, directory fuzzing, and manual exploitation. Learning how to capture live HTTP POST requests using Burp Suite, analyze headers, modify parameters in the Repeater, and document vulnerabilities using professional CVSS standards has greatly improved my technical skills.

This experience has built my confidence, and I am excited to apply these skills as I work toward my career goal of serving as a technical officer in the military forces. I am very grateful to the mentors at Dev ShieldX for structuring such a practical and comprehensive internship.

- Public GitHub Repository:

<https://github.com/IrsaAttiqueCyber/DevShieldX-Capstone-IrsaAttique>

- LinkedIn Project Showcase Post:

https://www.linkedin.com/posts/irsa-attique-a50aa4350_cybersecurity-penetrationtesting-owasp-activity-7483546697013420034-dJAU/?utm_source=social_share_send&utm_medium=member_desktop_web&rcm=ACoAAFe6AtwBnvA1tq5ExqYdlV36jxIOFejXwzs